

Cooperative Driving at Multiple Unsignalized Intersections in Fully Autonomous Driving Scenarios

Weihang Pan¹, Binbin Lin¹, Yafei Wang¹, Zhengxu Yu¹, Xinkui Zhao¹, Xiaofei He¹, *Senior Member, IEEE*, and Jieping Ye¹

Abstract—The decision-making process for connected and autonomous vehicles (CAVs) at unsignalized intersections is a critical and challenging problem. Previous methods predominantly concentrate on optimizing passage strategies for individual intersections in isolation. However, they often neglect global traffic conditions and task priorities in closed, multi-intersection transportation scenarios, leading to localized congestion. In this work, we propose a method that aims to optimize the passing order of intersections from a global and long-term perspective to enhance overall transportation efficiency. Specifically, we model the coordination of multiple unsignalized intersections as a multi-agent sequential decision problem and solve it through a two-stage method. In the planning stage, we construct fully connected undirected graphs based on vehicle conflict relationships and use the multi-agent proximal policy optimization (MAPPO) algorithm to learn the global priorities. In the scheduling stage, the local vehicle scheduling is formalized as a multi-objective optimization problem. The learned global priorities are soft constraints, while a hybrid filtered beam search determines safe and efficient CAV passing orders. Extensive offline experiments and online tests on real-world and synthetic datasets demonstrate that our proposed method outperforms state-of-the-art approaches in minimizing congestion and enhancing transportation efficiency.

Index Terms—Connected and automated vehicles (CAVs), cooperative driving, multiple unsignalized intersections, multi-agent reinforcement learning.

I. INTRODUCTION

COORDINATING large-scale connected and autonomous vehicles (CAVs) at multi-intersection scenarios is a

Received 7 November 2024; revised 18 April 2025 and 17 July 2025; accepted 18 September 2025. This work was supported in part by the Key Research and Development Program of Zhejiang Province under Grant 2025C01212, in part by the Yongjiang Talent Introduction Programme under Grant 2022A-240-G, in part by Ningbo Key Research and Development Program under Grant 2023Z229, and in part by the Scientific Research Fund of Zhejiang Provincial Education Department under Grant Y202457154. The Associate Editor for this article was Y. Zhou. (*Corresponding author: Binbin Lin.*)

Weihang Pan, Binbin Lin, and Xinkui Zhao are with the School of Software Technology, Zhejiang University, Hangzhou 310027, China (e-mail: panweihang@zju.edu.cn; binbinlin@zju.edu.cn; zhaoxinkui@zju.edu.cn).

Yafei Wang is with the Polytechnic Institute, Zhejiang University, Hangzhou 310027, China (e-mail: 22260466@zju.edu.cn).

Zhengxu Yu and Jieping Ye are with Alibaba Group, Hangzhou, Zhejiang 310027, China (e-mail: yuzxfred@gmail.com; yejieping.ye@alibaba-inc.com).

Xiaofei He is with the State Key Laboratory of CAD&CG, Zhejiang University, Hangzhou 310027, China (e-mail: xiaofeihe@cad.zju.edu.cn).

Digital Object Identifier 10.1109/TITS.2025.3615073

highly challenging problem. In the intelligent transportation systems, the dispatch center uses the real-time data of autonomous vehicles to coordinate the sequence of vehicles through intersections [1], [2].

Typically, Multi-agent Path Finding (MAPF) addresses conflict resolution at unsignalized intersections by collaboratively optimizing spatiotemporal trajectories for connected vehicles. However, in closed transportation systems such as ports, the combination of high CAV density, sparse intersections, and dynamic environmental changes makes it computationally prohibitive for MAPF to generate fully collision-free paths within practical time constraints [3], [4], [5]. This computational bottleneck stems from MAPF's NP-hard nature and its requirement for synchronous global optimization. Consequently, industrial implementations predominantly adopt a hierarchical architecture: classic graph search-based offline path planning (e.g., Dijkstra [7], A* [8]) integrated with online real-time scheduling specifically targeted at intersection conflict zones.

Current research on conflict zone scheduling primarily aims to minimize vehicle traversal time at isolated intersections, demonstrating efficacy in free-flow traffic conditions [6], [9], [10], [11], [12]. However, this local optimization approach often fails to achieve system-wide efficiency in closed or semi-closed collaborative environments.

In industrial scenarios, each vehicle follows specific tasks with dependencies. When vehicles arrive at task points out of sequence, they cannot proceed promptly to subsequent tasks, potentially causing congestion [13], [14]. Moreover, passage decisions at different intersections may affect one another, meaning that locally optimized intersections do not ensure global efficiency [15], [16]. For instance, as illustrated in Fig. 1, giving V2 priority to enter Zone1 may temporarily relieve local congestion, but it needs to let V4 vehicles pass through Zone2 first; otherwise, it may cause congestion. This action propagates delays downstream, causing vehicles V3, V5, and subsequent ones to queue in Zone2. Such inter-dependencies between conflict areas create cascading bottlenecks that existing algorithms fail to resolve, resulting in globally suboptimal scheduling despite local improvements.

To address these challenges, this study incorporates the spatiotemporal data of vehicle movements and task priorities to optimize vehicle passage across multiple intersections, aiming

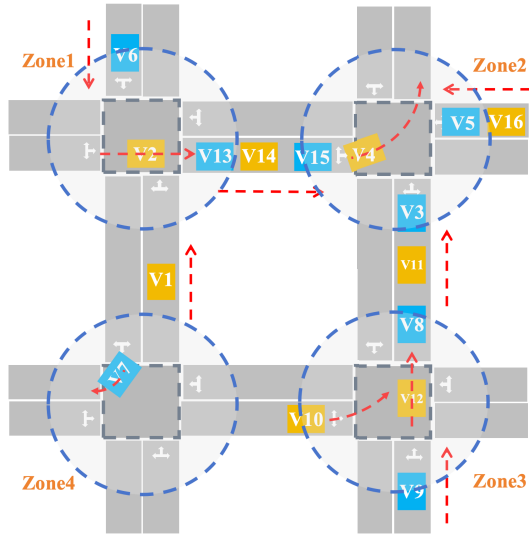


Fig. 1. Illustration of interdependent multiple intersections.

for enhanced overall efficiency. We model the conflict zone passage order as a multi-agent sequential decision-making problem, applying planning and scheduling methods to achieve coordinated optimization across the system.

Firstly, each unsignalized intersection is controlled by an individual agent, and a conflict graph is constructed based on the conflict relationships among CAVs. In the planning stage, we aim to optimize overall transport efficiency by training a neural network to determine the priority relationships between CAVs. Specifically, the decision-making process for establishing priority relationships is modeled as a Markov Decision Process (MDP) and trained using the Multi-Agent Proximal Policy Optimization (MAPPO) [17] algorithm. We model the interactions among CAVs using a Graph Neural Network (GNN) framework that leverages a message-passing mechanism [18]. In this framework, each CAV is represented as a node in the graph, and message passing is employed to capture the complex relationships within traffic scenarios. The outputs from the GNN serve as inputs to the safety layer, which performs priority correction and conflict resolution to generate feasible priority relationships for each CAV pair. In the scheduling stage, we aim to optimize vehicle coordination in real-time environments by prioritizing efficient and safe vehicle interactions. To achieve this, we use online optimization, leveraging learned global vehicle-to-vehicle (V2V) priority relationships as soft constraints to guide vehicle passage decisions dynamically. We model the V2V interaction optimization problem as a tree search, with each node representing a sequence of vehicle passages. To accelerate the search, we employ a hybrid filtered beam search method [19], [20] to approximate the optimal solution.

The main contributions are summarized as follows.

- To the best of our knowledge, we are the first to propose optimizing the passage order of CAVs among multiple unsignalized intersections in a fully autonomous transportation scenario from a global and long-term perspective.

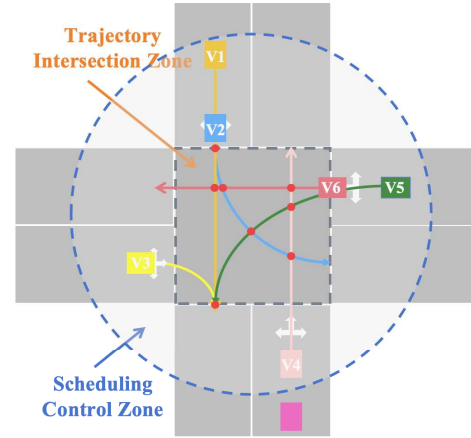


Fig. 2. A typical single intersection scenario. The area within the circle is referred to as the Scheduling Control Zone, while the actual intersection area is known as the Trajectory Interaction Zone. The red circles indicate conflict points where vehicles may collide.

- We introduce a novel two-stage planning and scheduling framework. In the planning stage, we innovatively use MAPPO to learn a decision-making strategy that determines the priority relationships between pairs of vehicles. In the scheduling stage, we treat these priority relationships as soft constraints, formulating a V2V interaction optimization problem and applying a hybrid filtered beam search to determine the final vehicle passage order.
- We propose the use of graph structures to represent interactions between vehicles, utilizing a Graph Neural Network (GNN) based on a message-passing mechanism to infer global priority relationships, effectively handling dynamic and complex interactions.
- Extensive offline experiments and online tests on one real-world and two synthetic datasets demonstrate that our proposed method outperforms state-of-the-art approaches in minimizing congestion and enhancing transportation efficiency.

II. RELATED WORK

A. Single Intersection Scenario

In CAVs scheduling research, single-intersection management remains a fundamental yet critical challenge. As illustrated in Fig. 2, typical intersection management systems employ spatial partitioning with a “scheduling control zone” for trajectory planning and a “trajectory interaction zone” for conflict resolution. Within these interaction zones, four primary vehicle trajectory patterns emerge: crossing (Fig. 3a), converging (Fig. 3b), diverging (Fig. 3c), and non-conflicting (Fig. 3d). The first three patterns require strict temporal separation of vehicle movements to prevent collisions, while the non-conflicting mode permits simultaneous passage. This fundamental framework underpins most current research on unsignalized intersection management.

Existing approaches to single-intersection scheduling primarily employ rule-based systems and optimization algorithms. Numerous studies have developed expert rules [10], [21], [22], [23], negotiation strategies [24], [25], and

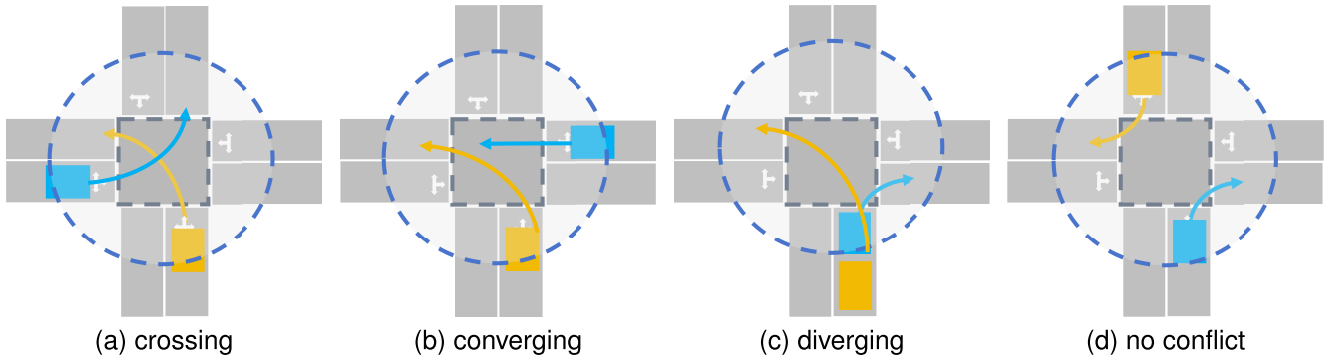


Fig. 3. Four typical interaction patterns of vehicle trajectories at intersections: Crossing, Converging, Diverging, and No Conflict. (a) The Crossing pattern indicates vehicles traveling on intersecting paths. (b) A Converging conflict occurs when vehicles enter the same lane. (c) A Diverging conflict indicates vehicles departing from the same lane. In the first three interaction cases, vehicles are not allowed to enter the intersection simultaneously. (d) In contrast, vehicles with No Conflict can enter the intersection simultaneously without the risk of collision.

reservation-based protocols [26], [27] for vehicle coordination. Although effective in controlled scenarios, these methods exhibit limited generalizability due to their reliance on human expertise, particularly in dynamic traffic environments [1], [2].

Mathematical programming approaches have provided alternative solutions through Mixed-Integer Linear Programming (MILP) [6], [28] and Convex Second-Order Cone Programming formulations [29]. Although these methods theoretically guarantee optimal solutions, their practical application is limited by computational complexity and real-time performance requirements [30].

Recent advancements focus on balancing computational efficiency with solution quality. Monte Carlo Tree Search (MCTS) variants [11], [12], [31], [32], [33], [34], [35], [36] and virtual queue-based algorithms [37] have demonstrated particular promise in this regard. Alternatively, deep reinforcement learning [38], [39] is developing into a new technical approach.

However, when extending single-intersection scheduling solutions to coordinated control of multiple conflict zones in industrial scenarios, due to the strict temporal dependencies of vehicle tasks, locally optimized traffic decisions trigger cascading congestion through coupled conflict zone pathways, ultimately resulting in a “local relief–global deterioration” scheduling paradox.

B. Multi-Agent Path Finding

MAPF addresses the problem of planning collision-free paths for multiple agents with distinct start and goal positions, with core constraints ensuring agents reach their targets without collisions while optimizing both solution quality and computational efficiency [3]. This field has been extensively studied through two primary algorithmic paradigms: centralized planning and distributed execution approaches.

Centralized approaches leverage global information for systematic conflict resolution. The Conflict-Based Search (CBS) [3] and its variants (e.g., Enhanced CBS [4], Lazy CBS [5], and Multi-Constraint CBS [40]) optimize paths through hierarchical constraint tree processing, while other methods like Increasing Cost Tree Search (ICTS) [41] employ two-phase decomposition. Machine learning-assisted

approaches such as MAPF-LNS2 [42] further enhance solution efficiency.

Distributed methods [43], [44], [45], [46] show greater promise in dynamic environments, particularly RL frameworks. Approaches like PRIMAL [43] and MAGAT [44] achieve decentralized coordination through hybrid learning strategies and communication mechanisms, though they face challenges like reward sparsity in partially observable scenarios [46].

While these MAPF approaches provide valuable insights, our study on unsignalized intersection scheduling addresses distinct challenges arising from fundamental differences between the problem domains. First, MAPF typically optimizes collision-free paths through global coordination, though this often proves infeasible in practice due to inevitable path conflicts. In contrast, intersection scheduling handles unavoidable conflicts by locally coordinating vehicle sequences at single conflict points with right-of-way rules. Second, intersection scheduling requires adherence to strict kinematic models and traffic regulations, contrasting with MAPF’s abstract grid-based movement assumptions. Third, our scenario demands millisecond-level trajectory synchronization for safe merging, necessitating continuous-time coordination rather than MAPF’s conventional discrete-timestep paradigm. These distinctions motivate our specialized approach, which combines motion prediction with priority-based negotiation rather than directly applying MAPF solutions.

III. METHODOLOGY

A. Problem Formulation

In this paper, we investigate the global scheduling problem for autonomous vehicle fleets in closed environments with multiple intersections, as illustrated in Fig. 4. We consider a directed road network $G = (V, E)$ with M heterogeneous conflict zones, where N autonomous vehicles execute transportation tasks under the following assumptions:

- Homogeneous vehicle automation capabilities.
- Lane-changing prohibition in control zones.
- Real-time V2V communication for sharing vehicle states (position, velocity) and intentions.

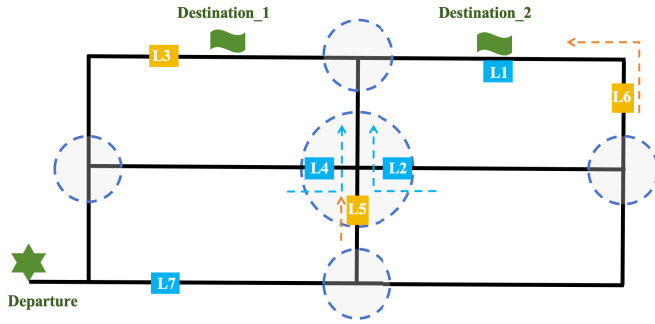


Fig. 4. Schematic of vehicle transportation tasks in an industrial setting. Each vehicle is assigned a specific task, departing from the starting point (Departure) to Destination_1 or Destination_2. After docking at the destination, the vehicle is reassigned a new task.

- All vehicles strictly follow pre-planned routes.

Each transportation task is defined as a pair $\mathcal{T}_i = (o_i, d_i)$, where $o_i, d_i \in V$ are the nodes of origin and destination. These tasks reside in a predefined ordered set $\mathcal{S} = \langle \mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_n \rangle$. The system follows a sequential allocation: upon completion of the current task $\mathcal{T}_k$, the next task $\mathcal{T}_{k+1}$ in the sequence will be automatically assigned. This sequential allocation mechanism dynamically generates a demand matrix for origin-destination (OD) $\mathbf{D}(t)$, where each element $D_{od}(t)$ represents the number of active transportation tasks from node o to node d at time t . The matrix evolves as tasks are completed and new ones are assigned from the predefined sequence.

The core challenge lies in constructing a decision-making framework that coordinates long-term global coordination with physical safety guarantees to achieve holistic optimization of the traffic system. To address this challenge, we propose a hierarchical decomposition approach that decouples the problem into two layers: 1) global priority planning for multi-intersection coordination and 2) real-time intersection safety scheduling.

The global priority planning is formulated as a constrained optimization problem with multiple hard constraints. As illustrated in Fig. 5, each vehicle is modeled as a node in a directed graph, where different edge types encode priority relationships and constraints:

- **Conflict edges** (E_{Conflict}): Vehicle pairs with intersecting paths.
- **Following edges** ($E_{\text{Following}}$): Predecessor vehicles in the same lane.
- **Fixed edges** (E_{Fixed}): Predefined priority relationships.
- **Prior edges** (E_{Prior}): Absolute priorities at intersections.

The global planner generates preference edges (E_{Prefer}) representing prioritized passing orders and must satisfy two fundamental constraints:

- 1) **Strong Priority Constraints:** For any vehicle pair (v_i, v_j) with the following relationships, predefined priorities, or intersection priorities, the precedence relationship must be strictly enforced:

$$\forall (v_i, v_j) \in E_{\text{Following}} \cup E_{\text{Fixed}} \cup E_{\text{Prior}}, \quad v_i < v_j \quad (1)$$

- 2) **Acyclicity Constraint:** The global priority relations $\mathcal{R} = \{<_{ij}\}$ must form a Directed Acyclic Graph (DAG)

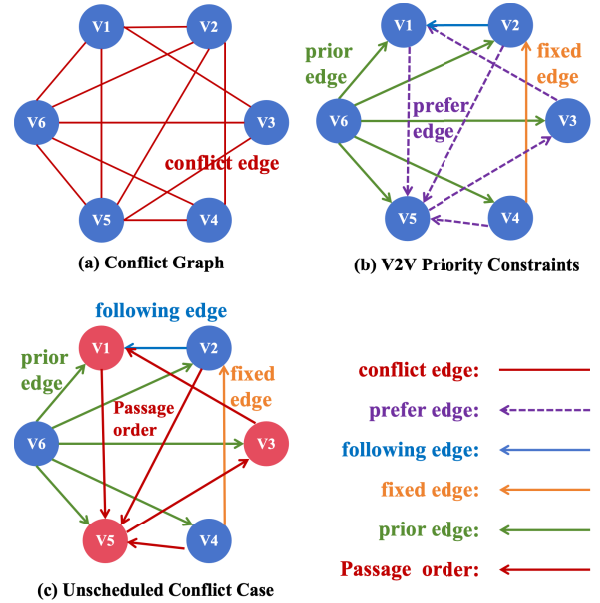


Fig. 5. Global priority planning problem formulation. Fig. (a) shows the Conflict Graph, where conflict edges (E_{Conflict}) represent pairs of vehicles with intersecting paths; Fig. (b) illustrates the vehicle priority relationship graph generated via V2V communication, which includes three types of hard constraint edges—the following edge ($E_{\text{Following}}$) giving priority to preceding vehicles in the same lane, predefined priority edges (E_{Fixed}), and absolute priority edges (E_{Prior}) at intersections; Fig. (c) presents a planning result that violates the Acyclicity Constraint, which will affect the subsequent solution of the scheduling optimization problem.

to prevent cyclic dependencies. For all preference edges E_{Prefer} , the following must hold:

$$\nexists \{v_1 < v_2 < \dots < v_k\} \text{ in } \mathcal{R} \cup E_{\text{Prefer}} \quad (2)$$

In Fig. 5-c, a directed cycle is formed among nodes v_1 , v_3 , and v_5 , violating the above constraint. This will affect the solution of subsequent scheduling optimization problems.

The local vehicle scheduling problem is formalized as a time-constrained multi-objective optimization problem that must reconcile system efficiency with global coordination preferences while satisfying the real-time 50ms computation requirement. This timing constraint originates from the industry-standard 100ms end-to-end response requirement (aligned with Mobileye's industry benchmarks), with 50ms specifically allocated for perception and planning to maintain total system latency within the 100ms safety threshold [47].

$$\min_{b_{ji}, t_i^o} \underbrace{\alpha \cdot \max_i t_i^o}_{\text{Efficiency}} + \beta \underbrace{\sum_{(i,j) \in E_{\text{Prefer}}} (1 - b_{ji})}_{\text{Global Priority Alignment}} \quad (3)$$

where t_i^o denotes vehicle v_i 's exit time from the intersection, $\max_i t_i^o$ represents the latest exit time of all vehicles, and $b_{ji} \in \{0, 1\}$ indicates the passing order: $b_{ji} = 1$ indicates v_j passes before v_i , aligning with the preferred order in E_{Prefer} . Conversely, $b_{ji} = 0$ means v_i passes before v_j . The objective function balances minimizing traffic bottlenecks (first term) with adherence to global priority suggestions (second term), where α and β weigh their relative importance. Hard

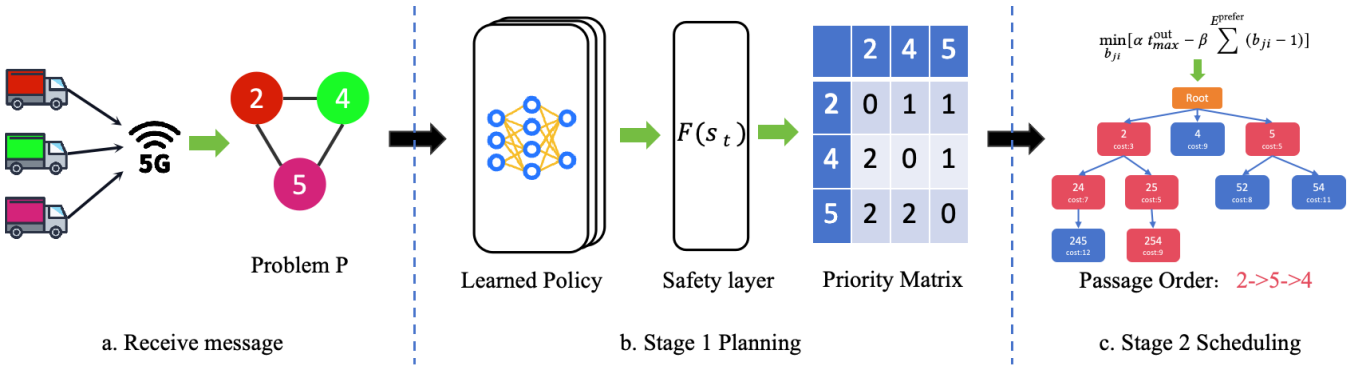


Fig. 6. Overall Framework. By modeling each intersection in the multi-intersection scenario as a V2V interaction optimization problem, a learned policy network is used to predict global priority preferences between vehicles. These preferences are then employed to guide the solution of the V2V interaction optimization problems.

constraints enforce collision avoidance, vehicle kinematics, and mandatory precedence rules (e.g., lane-following and predefined priorities), while global coordination preferences from E_{Prefer} are treated as soft constraints.

B. Overall Framework

We propose a two-stage algorithmic framework that integrates global spatiotemporal vehicle data, task information, and local passage order decisions (as illustrated in Fig. 6). In this framework, leveraging real-time passage information of all vehicles obtained through V2V communications, each CAV is modeled as a graph node. Undirected edges are established to represent potential conflict relationships between vehicles, which require assigned directions to indicate priority sequences. During the first stage, a trained policy network analyzes global traffic states to generate priority preferences among vehicles. The safety layer performs two critical operations: (1) hard constraints (e.g., $E_{Following}$) correct priority relationships, and (2) vehicle ID-based ordering resolves potential cyclic priority conflicts. The second stage employs a distributed scheduling module that dynamically determines passage sequences for CAVs at individual intersections based on the established priority information, ultimately achieving efficient collaborative management of traffic flows.

1) *Priority Preferences Plan*: In the planning stage, the trained policy network defines priority preferences for pairs of vehicles with potential conflicts based on global information. Specifically, the model outputs a priority matrix as a soft constraint, translating these preferences into directed preference edges that provide directional decisions for undirected conflict edges. Other directed edges (priority edges, following edges, and fixed edges) act as feasibility constraints. Section III-C provides a comprehensive explanation of how the policy network learns to predict global priority preferences between conflicting vehicles.

2) *Passage Order Schedule*: For each V2V interaction optimization problem, the scheduling module employs a hybrid filtered beam search algorithm to determine a near-optimal passage sequence among conflicting vehicles. This strategy achieves a well-balanced trade-off between computational efficiency and solution quality, ensuring that the calculation is

completed within 50ms. In Section III-D, we provide a detailed explanation of how the global priority preferences generated in the planning stage are used to calculate the actual passage order.

C. Planning

1) *Multi-Agent RL Setup*: Assuming that each control zone is controlled by an individual agent, we model the decision-making task of CAVs regarding task priority relationships as a Markov Decision Process (MDP), which consists of a series of states, actions, and rewards. Specifically, the MDP is composed of a tuple of six elements $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, as follows:

a) *State $\mathcal{S}$* : We have designed individual feature representations for each control zone, which include the control zone ID, the number of CAVs within the control zone, the number of lanes, the maximum number of queued CAVs, the minimum number of queued CAVs, and the number of CAVs for each destination, among others.

b) *Observation $\mathcal{O}$* : At time t , the state of CAVs in each control zone is defined as a two-dimensional vector $(n_v, n_{feature})$ representing the task and spatiotemporal status, where n_v indicates the number of CAVs, and $n_{feature}$ indicates the number of features. These features include task priority, road congestion levels, and the waiting conditions at the destination, among others.

c) *Action $\mathcal{A}$* : The agent's action $a_t \in \mathcal{A}$ involves categorizing each edge in the undirected graph G (the set of edges between CAVs in the control zone) into three main categories 0, 1, 2, where 0 indicates no relationship between the two nodes, 1 indicates that CAV A has priority over CAV B, and 2 indicates that CAV B has priority over CAV A.

d) *Reward $\mathcal{R}$* : We define the reward function $\mathcal{R}$ as the number of finished transportation tasks, given by the formula $\mathcal{R} = \sum_{i=1}^n V_i^{\text{task}}$, where V_i^{task} represents the number of tasks finished by vehicle i . The goal is to maximize the number of tasks finished within a certain period.

e) *Transition Probability $\mathcal{P}$* : The decision-making process is conducted in a rolling manner. For each time step, given the global state s_t and the decision made by the agent, the system generates the next state s_{t+1} according to the transition probability $P(s_{t+1}|s_t, a_t)$.

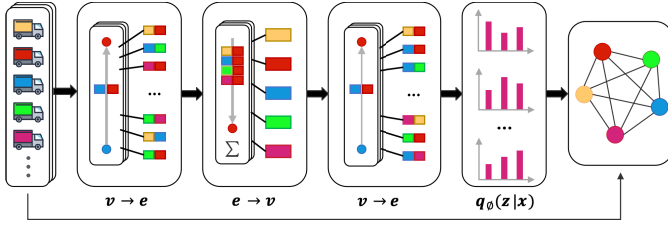


Fig. 7. The policy network is a message-passing graph neural network that infers CAV nodes' passing decision types z_{ij} through multiple rounds of node-to-edge ($v \rightarrow e$) and edge-to-node ($e \rightarrow v$) message passing.

f) *Discount Factor γ* : The discount factor controls how far into the future the MDP looks. In our application, it is beneficial to use a smaller discount factor because a longer horizon introduces greater variance in the value function.

In the framework of an MDP, a parameterized policy π_θ translates states into probability distributions of actions $\mathcal{P}(\mathcal{A})$, where θ represents the learnable parameters of the policy.

Besides the policy π_θ , to ensure that the priority relationship decisions comply with the constraints in equations (14)–(24), we have designed a feasibility rule $F(s_t)$, which, based on the current traffic conditions, determines the available actions at step t . Therefore, based on the policy $\pi_\theta(a_t|s_t)$ and the feasibility rule $F(s_t)$, the agent interacts with the environment to select the following action.

$$p(a'_t|s_t) = \pi_\theta(a_t|s_t) \circ F(s_t) \quad (4)$$

Here, the actions stored in the buffer are the corrected a'_t .

We aim to discover a policy that maximizes the expected cumulative reward, which is defined as follows:

$$J(\pi_\theta) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} [R(s_t, a_t)] \quad (5)$$

2) *Policy Network*: The overall transportation environment is dynamic, leading to complex and evolving interactions within the transportation system. We have access only to the individual task characteristics of each CAV, remaining unaware of the potential interactive behaviors, which correspond to the edges in the undirected, fully connected graph. We expect the policy network to learn the implicit relationships within the overall transportation system, thereby providing rational decisions on priority relations. Additionally, it should favorably prioritize decisions on the transit order that are more likely to enhance future transportation efficiency, assigning them a higher probability.

To achieve the aforementioned requirement, we employed a message-passing graph neural network [18] to parameterize our priority relationship decision strategy. As illustrated in Fig. 7, the objective of the policy network is to infer the pairwise passing decision types z_{ij} given the feature vectors $x^t = (x_1^t, \dots, x_N^t)$ of N CAV nodes at time step t . The feature vector of node i consists of components from T time parts: $x_i = (x_i^1, \dots, x_i^T)$. We model the policy network as $q_\phi(z_{ij}|x) = \text{softmax}(f_\phi(x)_{ij,1:N})$, where $f_\phi(x)$ is a GNN acting on the fully connected graph G , including functions such as $f_{\text{emb}}(x_j)$ for embedding node features, $f_e^1([h_i^1, h_j^1])$ for computing initial edge embeddings between node pairs, and $f_v(\cdot)$ for aggregating messages from neighboring nodes. Given

the node information $x_1, \dots, x_N$, our policy network computes the following message-passing operations.

$$h_j^1 = f_{\text{emb}}(x_j) \quad (6)$$

$$v \rightarrow e : h_{(i,j)}^1 = f_e^1([h_i^1, h_j^1]) \quad (7)$$

$$e \rightarrow v : h_j^2 = f_v\left(\sum_{i \neq j} h_{(i,j)}^1\right) \quad (8)$$

$$v \rightarrow e : h_{(i,j)}^2 = f_e^2([h_i^2, h_j^2]) \quad (9)$$

Ultimately, the policy network infers the priority relationships between nodes by applying a softmax function to the node embeddings $h_{(ij)}^2$ in conjunction with the parameters defined by neural networks in equations (6)–(9). Through multiple information-passing iterations, the network deciphers complex interactions. While in a single pass, the embedding relies only on data from directly connected node pairs, multiple passes allow it to integrate information from the entire graph. In our experiments, we employ the LSTM network for all functions $f(\dots)$.

3) *Training*: In this section, our goal is to train a policy network with parameter θ to maximize the expected cumulative reward as shown in equation (5). We employ MAPPO [17], where the Actor is synonymous with our policy network, used to generate a probability distribution over the preferences of CAVs. The Critic estimates the value function given the state, which measures the expected cumulative reward of the current policy in a particular state.

The actor network, denoted as π_θ , maps an agent's observations $o_i^{(a)}$ to a categorical distribution over actions in discrete action spaces, with all agents sharing the same policy network parameters. It is trained to maximize the following objective function:

$$L(\theta) = \frac{1}{Bn} \sum_{i=1}^B \sum_{k=1}^n \left[\min \left(r_{\theta,i}^{(k)} A_i^{(k)}, \text{clip}(r_{\theta,i}^{(k)}, 1 - \epsilon, 1 + \epsilon) A_i^{(k)} \right) \right] + \sigma \frac{1}{Bn} \sum_{i=1}^B \sum_{k=1}^n S[\pi_\theta(o_i^{(k)})] \quad (10)$$

where,

$$r_{\theta,i}^{(k)} = \frac{\pi_\theta(a_i^{(k)}|o_i^{(k)})}{\pi_{\text{old}}(a_i^{(k)}|o_i^{(k)})}, \quad (11)$$

B refers to the batch size, n refers to the number of agents. $A_i^{(k)}$ is calculated using the Generalized Advantage Estimation (GAE) method [48], S represents the policy entropy, and σ is the entropy coefficient hyperparameter.

The critic network is trained to minimize the loss function:

$$L(\phi) = \frac{1}{Bn} \sum_{i=1}^B \sum_{k=1}^n (\max[(V_\phi(s_i^{(k)}) - \hat{R}_i)^2, (\text{clip}(V_\phi(s_i^{(k)}), V_{\phi_{\text{old}}}(s_i^{(k)}) - \epsilon, V_{\phi_{\text{old}}}(s_i^{(k)}) + \epsilon) - \hat{R}_i)^2]) \quad (12)$$

where $\hat{R}_i$ is the discounted reward-to-go. For a detailed training algorithm, see Algorithm 1.

Algorithm 1 Training

```

1: Initialize  $\theta$ , the parameters for policy  $\pi$  and  $\phi$ , the parameters for critic  $V$ , using Orthogonal initialization
2: Set learning rate  $\alpha$ 
3: while step  $\leq \text{step}_{\max}$  do
4:   set data buffer  $D = \{\}$ 
5:   for  $i = 1$  to  $\text{batch\_size}$  do
6:      $\tau = \{\}$ 
7:     initialize  $h_{0,V}^{(1)}, \dots, h_{0,V}^{(n)}$  critic RNN states
8:     for  $t = 1$  to  $T$  do
9:       for all agents  $a$  do
10:         $p_{t,\pi}^{(a)}, h_{t,\pi}^{(a)} = \pi(o_t^{(a)}, h_{t-1,\pi}^{(a)}; \theta)$ 
11:        calculate hard rule  $F(s_t)$ 
12:         $u_t^{(a)} \sim p_t^{(a)} \circ F(s_t)$ 
13:         $v_{t,V}^{(a)}, h_{t,V}^{(a)} = V(s_t^{(a)}, h_{t-1,V}^{(a)}; \phi)$ 
14:      end for
15:      Execute actions  $u_t$ , observe  $r_t, s_{t+1}, o_{t+1}$ 
16:       $\tau \leftarrow [s_t, o_t, h_{t,\pi}, h_{t,V}, u_t, r_t, s_{t+1}, o_{t+1}]$ 
17:    end for
18:    Compute advantage estimate  $\hat{A}$  via GAE on  $\tau$ , using PopArt [49]
19:    Compute reward-to-go  $\hat{R}$  on  $\tau$  and normalize with PopArt [49]
20:    Split trajectory  $\tau$  into chunks of length  $L$ 
21:    for  $l = 0, 1, \dots, T/L - 1$  do
22:       $D = D \cup \{ \tau[l:l+L], \hat{A}[l:l+L], \hat{R}[l:l+L] \}$ 
23:    end for
24:  end for
25:  for mini-batch  $k = 1, \dots, K$  do
26:     $b \leftarrow$  random mini-batch from  $D$  with all agent data
27:    for each data chunk  $c$  in the mini-batch  $b$  do
28:      update RNN hidden states for  $V$  from first hidden state in data chunk
29:    end for
30:    Adam update  $\theta$  on  $L(\theta)$  with data  $b$  (equation (10))
31:    Adam update  $\phi$  on  $L(\phi)$  with data  $b$  (equation (12))
32:  end for
33: end while

```

D. Scheduling

1) *Real-Time Vehicle Scheduling Problem*: The objective of the online vehicle scheduling algorithm during each period is to minimize the maximum passage time of all vehicles within the decision radius as much as possible. In our study, this objective can also be interpreted as coordinating each vehicle to pass through conflict zones as quickly as possible while ensuring collision avoidance and meeting various physical constraints, thereby optimizing the global benefits in the future. The global priority preferences predicted between V2V pairs during the planning stage are incorporated as penalty terms in the objective function.

Formally, the objective function of the centralized vehicle traffic scheduling algorithm is written as follows:

$$\min_{b_{ji}} \left[\alpha t_{\max}^{\text{out}} + \beta \sum_{E^{\text{prefer}}} (1 - b_{ji}) \right] \quad (13)$$

$$\text{s.t. } t_{\max}^{\text{out}} \geq t_i^o, \quad \forall i \in \mathcal{V} \quad (14)$$

$$t_i^{\text{in}} \geq t_i^p, \quad \forall i \in \mathcal{V} \quad (15)$$

$$t_i^o \geq t_i^{\text{in}} + \Delta t_i, \quad \forall i \in \mathcal{V} \quad (16)$$

$$t_i^p \geq t_j^{\text{in}} + \Delta t_j^d, \quad \forall i, j \in E^{\text{following}} \quad (17)$$

$$t_i^p \geq t_i^{\text{p-min}} + (t_j^{\text{in}} - t_j^p), \quad \forall i, j \in E^{\text{following}} \quad (18)$$

$$t_i^{\text{in}} \geq t_j^o, \quad \forall i, j \in E^{\text{following}} \quad (19)$$

$$b_{ij} + b_{ji} = 1, \quad \forall i, j \in E^{\text{conflict}} \quad (20)$$

$$b_{ij} \cdot M + t_i^{\text{in}} \geq t_j^o, \quad \forall i, j \in E^{\text{conflict}} \quad (21)$$

$$b_{ji} = 1, \quad \forall i, j \in E^{\text{fixed}} \quad (22)$$

$$b_{ji} = 1, \quad \forall i, j \in E^{\text{prior}} \cap E^{\text{conflict}} \quad (23)$$

$$t_j^{\text{in}} = t_j^p = t_j^{\text{fact}}, \quad \forall j \in E^{\text{prior}} \quad (24)$$

The symbols used above are summarized in TABLE I.

Equation (14) defines the lower bound for the optimization objective $t_{\max}^{\text{out}}$ in Equation (13).

Following the formulation of the objective function and constraints, we elucidate the constraints' rationale within our proposed vehicle scheduling model. The constraints of our model are designed to ensure the smooth flow of vehicles through these interaction points.

Equations (15), (16) indicate that for each node, the state time variables of each vehicle have a fixed sequential relationship. Equations (17), (18), and (19) indicate that for each following edge, the trailing vehicle must wait until the leading vehicle has fully entered the intersection before it can proceed. The actual time at which the trailing vehicle arrives at the intersection is adjusted based on the delay of the leading vehicle, and the trailing vehicle can only start to move through the intersection once the leading vehicle has completely passed. Equation (20), (21) indicate that for each conflict edge, the priority relationship between two conflicting nodes is represented by a pair of decision variables, which sum to 1. The vehicle with a lower priority must wait until the higher-priority vehicle has completely exited the intersection before it can enter. Equations (22) indicate that for each fixed edge, the corresponding decision variable is initialized to 1, reflecting the predetermined priority relationship. Equations (23), (24) indicate that for each prior edge, the time node of the prior node requires special calculation, and the associated decision variable is set to 1.

Through this intricate network of variables and constraints, our model seeks to promote globally efficient collaborative management of traffic flows, balancing delays, safety, and adherence to scheduled routes.

2) *Hybrid Filtered Beam Search*: Using a passage sequence, we represent the solution to the V2V interaction optimization problem. For example, the solution $v2 \rightarrow v5 \rightarrow v4$ indicates that vehicles $v2$, $v5$, and $v4$ pass through the same conflict area in that order. The solution space grows factorially with the number of vehicles, as there exist $n!$ possible permutations for n vehicles.

To address this combinatorial explosion, we employ a Hybrid Filtered Beam Search algorithm (Fig. 8, Algorithm 2) that combines breadth-first search with a candidate selection mechanism incorporating temporal coherence preservation.

TABLE I
LIST OF NOTATIONS

Index Symbols		
Symbol	Index	Description
$i, j \in \mathcal{V}$	/	Node index in the conflict graph.
Set Symbols		
Symbol	Index	Description
$\mathcal{V}$	/	The set of nodes in the conflict graph, representing all vehicles.
E^{conflict}	/	The set of directed node pairs (j, i) in conflict edges, corresponding to vehicle pairs with conflicts.
E^{prefer}	/	The set of directed node pairs in preference edges, representing preferred priority relationships $(j \rightarrow i)$.
E^{prior}	/	The set of directed node pairs in prior edges, indicating absolute priority of vehicles partially or fully in the intersection $(j \rightarrow i)$.
$E^{\text{following}}$	/	The set of directed node pairs in following edges, representing sequential relationships within the same lane $(j \rightarrow i)$.
E^{fixed}	/	The set of fixed edges, representing preassigned priority relationships $(j \rightarrow i)$.
Decision Variables Symbols		
Symbol	Index	Description
b_{ji}	$\forall i, j \in E^{\text{conflict}}$	A binary variable indicating whether vehicle j passes before vehicle i . If j has priority over i , then $b_{ji} = 1$; otherwise, $b_{ji} = 0$.
$t_i^p \in \mathcal{Q}$	$\forall i \in \mathcal{V}$	The actual arrival time of vehicle i at the intersection.
$t_i^{\text{in}} \in \mathcal{Q}$	$\forall i \in \mathcal{V}$	The actual time when vehicle i starts entering the intersection.
$t_i^o \in \mathcal{Q}$	$\forall i \in \mathcal{V}$	The time when vehicle i leaves the intersection.
Constant Variables Symbols		
Symbol	Index	Description
$t_i^{\text{p-min}}$	$\forall i \in \mathcal{V}$	The predicted minimum arrival time of vehicle i at the intersection.
t_j^{fact}	$\forall i \in \mathcal{V}$	The actual arrival time of vehicle j (used for prior edges).
Δt_i	$\forall i \in \mathcal{V}$	The time duration required for vehicle i to pass through the entire intersection without stopping.
Δt_i^d	$\forall i \in \mathcal{V}$	The time duration required for vehicle i to completely enter the intersection.
Objective Function Symbols		
Symbol	Index	Description
$t_{\text{max}}^{\text{out}}$	/	The maximum departure time of all vehicles, used to measure overall travel time optimization.
α	/	A weight parameter in the objective function, balancing the maximum departure time.
β	/	A weight parameter in the objective function, balancing penalties for preference edges.
Other Symbols		
Symbol	Index	Description
M	/	A large number used to enforce priority constraints.
$\mathcal{Q}$	/	Indicates that the variable belongs to the set of rational numbers.

The algorithm first extracts a fixed prefix s_{fix} from the previous timestep's solution to ensure short-term decision consistency. The length n of the fixed prefix is dynamically determined as $n = \max(k, m)$, where k denotes the number of vehicles already occupying the conflict zone in the previous timestep, and m represents the number of lanes containing approaching vehicles in the previous timestep. This design embeds historical ordering information into the new solution generation process, preventing abrupt changes in vehicle passage sequences. During the iterative search from level $n + 1$ to d_{max} (where d_{max} represents the total number of vehicles), four core operations are performed at each level: 1) **Child**

Algorithm 2 Hybrid Filtered Beam Search for V2V Interaction Optimization

Require:

- 1: S_{prev} : Previous timestep's ordered sequence
- 2: b : Beam width
- 3: d_{max} : Number of vehicles to sequence
- 4: n : Fixed prefix length ($\max(\text{occupied vehicles } k, \text{ active lanes } m)$)
- 5: $G = (V, E_{\text{Following}}, E_{\text{Fixed}}, E_{\text{Prior}})$: Constraint graph

Ensure: Stable passing sequence S_{new}

- 6: Initialize fixed prefix $s_{\text{fix}} \leftarrow S_{\text{prev}}[1 : n]$
- 7: Initialize queue S_{best} with s_{fix} as root
- 8: **for** level = $n + 1$ to d_{max} **do**
- 9: Generate all child nodes from the current queue
- 10: Prune nodes violating feasibility constraints in G
- 11: **for** each valid node s_i **do**
- 12: Evaluate objective function $f(s_i)$
- 13: **end for**
- 14: Sort nodes by composite score $f(s_i)$
- 15: Retain top- b candidates
- 16: Update queue with selected nodes
- 17: **end for**
- 18: Select $S_{\text{best}} \leftarrow \arg \max f(s_i)$ from final queue
- 19: **return** S_{best}

node generation: Appending valid vehicles to existing partial sequences; 2) **Constraint pruning:** Eliminating candidate solutions that violate the directed edges of graph G to ensure strict feasibility; 3) **Objective evaluation:** Computing the objective function value of the passage order represented by the node; and 4) **Greedy selection:** Retaining the top- b candidate solutions to balance exploration and exploitation.

This hybrid strategy achieves computational tractability through two key mechanisms: The beam width b controls the intensity of solution space exploration, while the fixed prefix s_{fix} ensures temporal stability by anchoring previously optimized partial sequences. The algorithm exhibits $O(b \times (d_{\text{max}} - n) \times m)$ linear time complexity, where m denotes the time required to evaluate a node. As demonstrated in Section IV, this design proves effective in real-world traffic scenarios.

IV. EXPERIMENTS

A. Simulation

The experimental environment was constructed based on the open-source microscopic traffic simulation platform SUMO (Simulation of Urban Mobility) [50]. Using SUMO's built-in netedit tool, we manually developed refined models for three typical scenarios—port, campus, and warehouse. The simulation platform achieved real-time interaction between SUMO and Python control scripts via the TraCI interface, with a time step of 1.0 seconds. Traffic signals were configured as “none” type, and the default conflict resolution mechanism was disabled, thus enabling cross-intersection control through external algorithms.

For vehicle behavior modeling, the Intelligent Driver Model (IDM) was adopted as the car-following model. The model

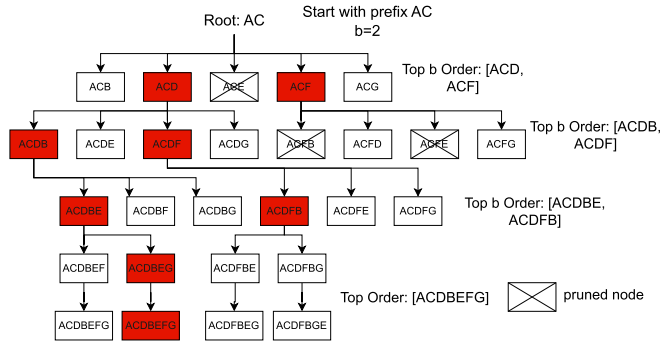


Fig. 8. Illustration of passage order resolution via hybrid filtered beam search ($b = 2$, prefix="AC").

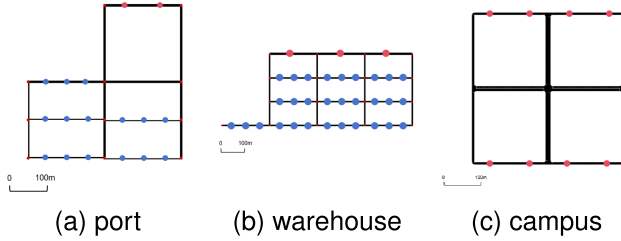


Fig. 9. The road networks of three datasets.

parameters include a maximum acceleration of 3.0 m/s^2 , a maximum speed of 15.0 m/s , and lane-changing prohibition (laneChangeMode = 0). Safety considerations were implemented through a minimum stopping distance of 3.5 m from leading vehicles, with normal braking deceleration of 4.5 m/s^2 and emergency deceleration capability up to 9.0 m/s^2 . However, it should be noted that actual autonomous vehicles in operational scenarios demonstrate more complex behaviors in trajectory planning and emergency response compared to the IDM representation. For CAV task scheduling, buffer queue mechanisms were implemented at loading/unloading points, where the system automatically assigns the next task from the buffer pool upon completion of the current task.

Optimal driving paths were determined using SUMO's built-in Dijkstra algorithm, which calculates the shortest path based on estimated travel time and real-time traffic conditions, with road segments as edge weights and intersections as nodes.

In the communication mechanism simulation, the packet loss rate was set to 0% with zero transmission delay. For intersection management, the control range was defined as 60 meters upstream of intersections, where detectors trigger control instructions.

B. Datasets

We conducted experiments on three different datasets representing three distinct scenarios, with each scenario containing a single instance. For the port dataset, which is based on a real port scenario, as shown in Fig. 9a, the road lengths range from 100 to 300 meters. This dataset contains 60 vehicles, with an OD matrix derived from an actual container ship's task pool. The transport task involves moving containers from storage

areas to berths for ship loading. The top horizontal road in the figure is designated as the berth (marked with red points), with two loading and unloading points. The bottom horizontal road contains five container areas (marked with blue points), each with three loading and unloading points.

Fig. 9b describes an automated warehouse transport scenario where road lengths also range from 100 to 200 meters. The warehouse scenario deploys 30 vehicles with a manually designed OD matrix, simulating express parcel delivery tasks from shelves to designated exits. The top three horizontal edges in the figure each have a packaging conveyor belt (marked with red points). The bottom section includes 10 shelves, each with three picking points (marked with blue points).

Fig. 9c represents a simplified campus autonomous vehicle shuttle scenario. All roads are 200 meters long. With 20 vehicles operating on fixed-route OD paths (all marked with red points), the task focuses on passenger transportation along predefined circular routes. Each CAV travels along a predetermined route in a loop.

In this study, we set up a conflict scheduling area at each intersection, centered on the intersection's midpoint, with a radius of 60 meters. We set the simulation time to 1200 steps, with each step corresponding to 1 second in real time, totaling 20 minutes. The code and datasets will be open-sourced at <https://github.com/pwhjy/MultiX-CoDrive>.

C. Evaluation Metrics

In this paper, we selected the following three metrics to evaluate performance:

- **Average Delay Time:** The average total delay time of n CAVs refers to the average of the total time lost by all vehicles due to traffic control, road congestion, and other reasons during their travel.

$$T_{\text{delay}} = \frac{\sum_{i=1}^n (T_{\text{actual}} - T_{\text{ideal}})}{n} \quad (25)$$

where T_{delay} represents the average total delay time, T_{actual} is the actual travel time, T_{ideal} is the ideal travel time.

- **Average Waiting Time:** The average waiting time of n vehicles refers to the average waiting time caused by vehicle queuing at intersections or other places where vehicles need to wait.

$$T_{\text{waiting}} = \frac{\sum_{i=1}^n T_{\text{wait},i}}{n} \quad (26)$$

where T_{waiting} represents the average waiting time, $T_{\text{wait},i}$ is the waiting time for vehicle i .

- **Number of Tasks Finished:** The number of tasks finished refers to the total number of successful tasks finished by vehicles within a certain time frame.

$$N_{\text{task}} = \sum_{i=1}^n V_i^{\text{task}} \quad (27)$$

where N_{task} is the total number of tasks finished, and V_i^{task} represents the number of tasks finished by vehicle i .

- **Time-Consuming:** The average time required for each algorithm to make a single decision.

D. Baselines

We compared several representative baseline models for CAV conflict scheduling at unsignalized intersections, including the following:

- **Random within Hard Rule:** This method randomly selects a legal passing order that complies with strict safety rules, such as maintaining safe distances and avoiding conflicts. The result is a safe crossing order.
- **SUMO RULE [50]:** In this approach, the vehicle passing order follows specific priority rules. For four-way stops, a “first-come, first-served” rule is applied, while at intersections between main and side roads, vehicles on the main road have the right of way.
- **FIFO [27]:** The standard FIFO (first-in, first-out) method determines the crossing order based on vehicle arrival time at the intersection. CAVs entering the control zone earlier are given higher priority in resolving conflicts. This event-driven approach updates the crossing sequence whenever a CAV enters or exits the control zone.
- **Modified FIFO [51], [52]:** This version of FIFO dynamically adjusts the crossing priority by assigning higher priority to CAVs closer to the conflict zone, updating the crossing sequence periodically based on the current distances of all CAVs from the zone.
- **Dynamic Resequencing (DR) [51], [52]:** The DR method reevaluates and adjusts the crossing sequence each time a new CAV enters the control zone. By inserting each new arrival into an optimal position in the sequence, this approach aims to improve overall traffic efficiency. The sequence is recalculated only when new vehicles arrive, making it event-driven.
- **Monte Carlo Tree Search (MCTS) [31], [34], [35]:** Using a tree-based representation of the solution space, MCTS combines the Monte Carlo tree search with heuristic rules to identify a near-optimal passing order within a short planning time.
- **Parallel Neural Monte Carlo Tree Search (PNMCTS) [36]:** PNMCTS enhances traditional MCTS by integrating neural networks and parallelized search strategies. It transforms unsignalized intersection scheduling into a board-game-like problem, enabling efficient exploration of the solution space.
- **DRL-LSTM:** This method follows a two-stage approach, where the first stage assigns a priority weight to each vehicle rather than determining priority relationships between pairs of vehicles. The priority weights guide the scheduling decisions in the second stage.

E. Implementation of Baselines

For fair comparison, all baselines are adapted to our problem setting with the following unified constraints:

- **Common safety rules:** All methods adhere to the same safety rules, such as maintaining a minimum safety distance $d_{\text{safe}} = 3.5\text{m}$ to prevent collisions.
- **Same control zone definition:** The control zone radius $R_c = 60\text{m}$ is consistently applied across all methods,

ensuring that the spatial scope of decision-making is identical.

- **Identical vehicle dynamics model:** All baselines use the same vehicle dynamics model to ensure consistency in how vehicle movements are simulated and evaluated.

Specific adaptations for each baseline are detailed below:

- **Random within Hard Rule:** At each decision step, if new vehicles are added, randomly sample a valid permutation from the solution space constrained by E^{prior} , $E^{\text{following}}$, and E^{fixed} ; otherwise, no resampling is triggered.
- **SUMO Rule:** We removed all additional controls and directly ran the experiment using SUMO’s default simulation rules.
- **FIFO:** We maintain a list S to store the CAVs currently in the control zone along with their entry timestamps. When a new CAV enters the control zone, its entry timestamp t is recorded, and it is appended to the end of the list S . When a CAV exits the conflict zone, it is removed from list S . The crossing sequence is determined by sorting the CAVs in ascending order based on their entry timestamps t , meaning that CAVs that enter earlier are given priority to pass first. This strategy is event-driven, as updates are triggered only when a CAV enters or exits the control zone.
- **Modified FIFO:** We maintain a list S to store the CAVs currently in the control zone, along with their Euclidean distances to the conflict zone centroid. The distance $d_i(t)$ of each CAV to the conflict zone centroid is computed in real time, and the crossing sequence is determined by sorting the CAVs in ascending order based on their distances $d_i(t)$, meaning that CAVs closer to the conflict zone are given priority to pass first. This strategy is time-driven, as the priority updates occur periodically every $T_{\text{update}} = 1.0\text{s}$ to ensure real-time adjustments.
- **DR:** We maintain the original crossing sequence S and dynamically adjust it to accommodate new CAVs while optimizing the overall crossing efficiency. The strategy proceeds as follows: First, the preceding vehicle of the new CAV and its position k in S are identified. Then, for each position i from the end of S to k , the new CAV is inserted at position $i + 1$, generating a feasible crossing sequence S_f . The corresponding objective value J_f is computed for S_f . If J_f is less than the current optimal value J_{optimal} , both J_{optimal} and the new crossing sequence S_{new} are updated. This iterative process ensures that the insertion of the new CAV does not degrade the overall crossing efficiency. Finally, the optimized crossing sequence S_{new} is returned. This strategy is event-driven, as updates are triggered only when a new CAV enters the control zone.
- **MCTS:** We enhance search efficiency by incorporating heuristic rules into the classic MCTS framework, with a 5-second computation budget allocated to ensure sufficient exploration. The implemented pseudo code is detailed in Algorithm 3.
- **PNMCTS:** We adapt the original PNMCTS framework (designed for platoon-based scheduling) to our

multi-intersection scenario via two critical modifications: (1) replacing platoon coordination with individual CAV scheduling to enable fine-grained control, and (2) introducing hard constraints ($E_{\text{Following}}$, E_{Fixed} , E_{Prior}) for vehicle-level conflict resolution. As detailed in Algorithm 4, the modified framework operates through three consecutive phases. The dynamic state update phase merges newly arrived CAVs while locking actions for vehicles already in conflict zones, with adjustable vehicles including both new arrivals and those approaching but not yet entering conflict areas. During the constrained MCTS search phase, valid actions are generated under the three hard constraints using a UCB-balanced tree search that integrates policy network guidance (π_θ) with historical value estimates, where terminal states are evaluated through weighted rewards considering both total crossing time and scheduling steps. Finally, the action selection phase chooses the optimal action based on root node visit counts before updating the board state accordingly. The policy network π_θ maintains the original training paradigm with identical architecture and curriculum learning strategy, while implementation specifics remain consistent with the baseline methodology.

Algorithm 3 MCTS for V2V Interaction Optimization

Require: State information of all CAVs

Ensure: Optimal crossing sequence S_{best}

```

1: Initialize root node with initial intersection state
2: while computational budget not exhausted do
3:   Selection Phase:
4:   Traverse from root using UCB1 policy
5:   Prune invalid branches with hard constraints
6:   Expansion Phase:
7:   Generate all valid child nodes satisfying:
8:    $E_{\text{Following}}, E_{\text{Fixed}}, E_{\text{Prior}}$  constraints
9:   Randomly select one valid node to expand
10:  Simulation Phase:
11:  for Monte Carlo sampling iteration do
12:    Generate complete feasible sequence via:
13:    Constraint propagation on partial sequence
14:    Calculate traffic delay & throughput metrics
15:    if current metrics surpass  $S_{\text{best}}$  then
16:      Update  $S_{\text{best}} \leftarrow$  current sequence
17:    end if
18:  end for
19:  Backpropagation Phase:
20:  Update node visit counts along path
21:  Propagate cumulative reward metrics
22: end while
23: return  $S_{\text{best}}$ 

```

- **DRL-LSTM:** The DRL-LSTM baseline adopts an identical hierarchical architecture to our proposed framework but diverges in the priority determination mechanism. In its first stage, rather than explicitly modeling pairwise interactions via GNNs, DRL-LSTM utilizes an LSTM to generate per-vehicle scalar priority weights directly from sequential observations of individual CAV states.

Algorithm 4 PNMCTS for V2V Interaction Optimization

Require: Initial board state s_{init} , New vehicle set $\mathcal{V}_{\text{new}}$ (can be empty), Policy network π_θ , Search iterations K , Hard rules $\{E_{\text{Following}}, E_{\text{Fixed}}, E_{\text{Prior}}\}$

Ensure: Optimal action a^* and updated state s_{new}

```

1: Phase 1: State Update
2: if  $\mathcal{V}_{\text{new}} \neq \emptyset$  then
3:    $s_{\text{new}} \leftarrow s_{\text{init}} \cup \mathcal{V}_{\text{new}}$ 
4:    $\mathcal{V}_{\text{locked}} \leftarrow \{v \mid v \text{ has entered conflict zone}\}$ 
5:    $\mathcal{V}_{\text{adjustable}} \leftarrow \mathcal{V}_{\text{new}} \cup \{v \in s_{\text{init}} \mid v \notin \mathcal{V}_{\text{locked}}\}$ 
6: else
7:    $s_{\text{new}} \leftarrow s_{\text{init}}, \mathcal{V}_{\text{adjustable}} \leftarrow$  all unlocked vehicles
8: end if
9: Phase 2: Priority-Constrained MCTS Search
10: Initialize MCTS tree with root  $s_{\text{root}} \leftarrow s_{\text{new}}$ 
11: for  $k \leftarrow 1$  to  $K$  do
12:    $s_{\text{current}} \leftarrow s_{\text{root}}$ 
13:   while  $s_{\text{current}}$  is not a leaf node do
14:     Generate valid actions under priority rules:
15:     
$$\mathcal{A}_{\text{valid}} \leftarrow \left\{ a \mid \begin{array}{l} a \text{ acts on } v \in \mathcal{V}_{\text{adjustable}}, \\ a \text{ satisfies } E_{\text{Following}}, E_{\text{Fixed}}, E_{\text{Prior}} \end{array} \right\}$$

16:      $a_{\text{select}} \leftarrow \arg \max_{a \in \mathcal{A}_{\text{valid}}} \left( Q + c_{\text{puct}} \cdot \pi_\theta \cdot \frac{\sqrt{N}}{1+N} \right)$ 
17:      $s_{\text{current}} \leftarrow \text{Execute}(a_{\text{select}})$ 
18:   end while
19:   if  $s_{\text{current}}$  is terminal then
20:      $v \leftarrow \alpha(1 - t_{\text{cross}}) + (1 - \alpha)(1 - \text{steps})$ 
21:   else
22:      $(p, v) \leftarrow \pi_\theta(s_{\text{current}})$ 
23:     Expand node with  $\mathcal{A}_{\text{valid}}$ 
24:   end if
25:   while backpropagation not done do
26:     Update  $N \leftarrow N + 1, Q \leftarrow Q + \frac{v - Q}{N}$ 
27:   end while
28: end for
29: Phase 3: Action Selection
30:  $a^* \leftarrow \arg \max_{a \in \mathcal{A}_{\text{valid}}} N(s_{\text{root}}, a)$ 
31:  $s_{\text{new}} \leftarrow \text{Apply Action}(s_{\text{new}}, a^*)$ 
32: return  $a^*, s_{\text{new}}$ 

```

F. Main Result

As shown in Table II, our method significantly outperforms all baseline models across three scenarios: port, warehouse, and campus, with each value representing the average of ten experimental runs. Taking the warehouse scenario as an example, compared with the best baseline MCTS algorithm, our method achieves an increase of 21 completed tasks (613 vs. 592, representing a 3.5% relative improvement), demonstrating our approach's capability to optimize overall decision-making from a long-term and global perspective.

Notably, the state-of-the-art baseline PNMCTS demonstrates performance limitations under our defined heterogeneous multi-intersection scenarios. This primarily stems from three factors: First, the PNMCTS model's independent decision-making mechanism, homogeneous training data, and fixed input features are fundamentally inadequate for modeling dynamic conflict rules at heterogeneous intersections and

TABLE II
THE OVERALL PERFORMANCE OF DIFFERENT MODELS ON THREE DATASETS. ALL-TIME METRICS ARE IN SECONDS

Data	Method	Avg Waiting Time	Avg Waiting Time	Task Finished	Time-Consuming
Port	Random	453.521	397.711	248	<0.0001
	Sumo Rule	441.316	413.000	251	<0.0001
	FIFO	443.149	382.444	256	<0.0001
	Modified FIFO	441.876	379.598	259	<0.0001
	DR	441.423	378.942	262	<0.0001
	MCTS	439.558	377.933	265	5.042
	PNMCTS	440.926	378.562	265	0.052
	DRL-LSTM	445.367	379.432	253	0.008
	Ours	409.346	352.933	272	0.012
Warehouse	Random	259.372	299.833	545	<0.0001
	Sumo Rule	237.935	309.217	573	<0.0001
	FIFO	219.642	240.517	581	<0.0001
	Modified FIFO	213.468	239.946	586	<0.0001
	DR	209.725	237.367	589	<0.0001
	MCTS	203.956	235.600	592	4.013
	PNMCTS	203.572	235.358	592	0.050
	DRL-LSTM	217.333	242.745	587	0.005
	Ours	182.987	215.650	613	0.008
Campus	Random	298.236	345.000	248	<0.0001
	Sumo Rule	285.918	315.429	258	<0.0001
	FIFO	271.905	309.429	264	<0.0001
	Modified FIFO	269.933	313.405	265	<0.0001
	DR	268.120	315.416	265	<0.0001
	MCTS	266.885	320.571	267	2.988
	PNMCTS	268.235	318.476	266	0.051
	DRL-LSTM	282.738	327.362	256	0.003
	Ours	256.668	308.571	272	0.005

achieving global coordination. Second, the PNMCTS method remains myopic as it solely focuses on optimizing traffic throughput at individual intersections rather than considering global optimization. Third, under our experimental configuration where MCTS is granted sufficient time budget for deep searches (achieving near-theoretical-limit performance), PNMCTS is inherently bounded by this optimization ceiling.

Compared with the LSTM-based direct prediction method for priority weights (DRL-LSTM), our algorithm achieves an absolute improvement of 26 completed tasks (613 vs. 587, 4.4% relative improvement). This significant enhancement primarily benefits from our introduced message-passing mechanism, which effectively propagates information through graph structures, enabling each node (CAV) to integrate information from other nodes. This global information integration allows the model to comprehend complex global scheduling relationships better, thereby making more rational priority decisions.

Using a kernel density estimation approach, we visualized the vehicle distribution at a specific moment in the warehouse dataset, as shown in Fig. 10. This method estimates the density of vehicles at each point in the two-dimensional warehouse space, with bandwidth adjustments controlling the smoothness of the density estimation. The result is a continuous color-gradient heatmap that intuitively highlights areas with varying

vehicle concentrations. It can be seen that the vehicle distribution at each intersection is relatively balanced. In contrast, the MCTs algorithm, for example, tends to result in certain intersections having more vehicles. In Fig. 11, we selected the number of vehicles on each lane over 140 steps. It can be observed that there is essentially no congestion, whereas the optimal baseline, MCTs, shows a certain degree of congestion. This indicates that our method can optimize from a global perspective, leading to a more even distribution of vehicles across intersections and avoiding systemic congestion caused by excessive local optimization.

G. Computational Complexity Analysis

To validate the real-time applicability of the scheduling algorithms, we conducted a comparative analysis between the exact solution method based on Mixed Integer Linear Programming (MILP) and the Hybrid Filtered Beam Search approach. The test scenarios included synthetic traffic environments with 5 to 20 vehicles, and the key metrics covered solution time and optimality gap.

As shown in Table III, for small-scale problems (number of vehicles $N \leq 10$), the CPLEX solver (version 22.1.1.2) achieved provable optimality by exhaustively searching the

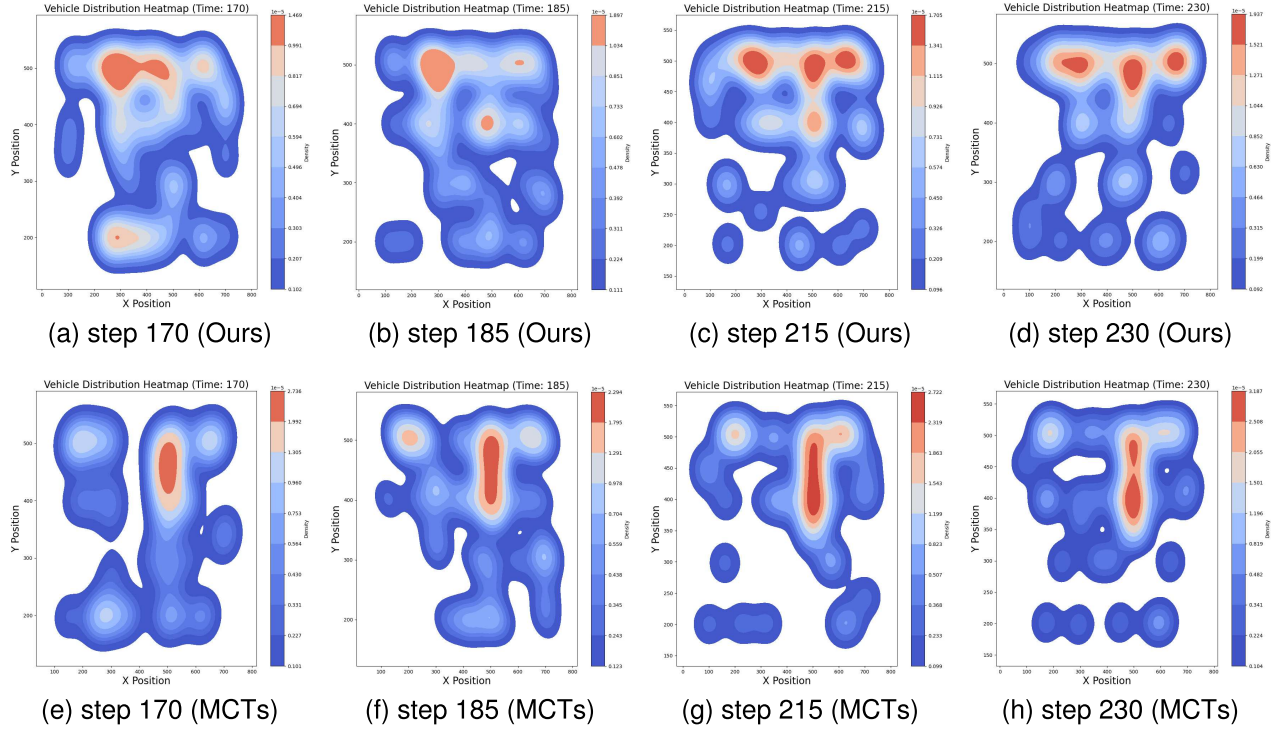


Fig. 10. Comparison of CAV distribution heatmaps at different time steps using our method (top row) and the MCTs algorithm (bottom row).

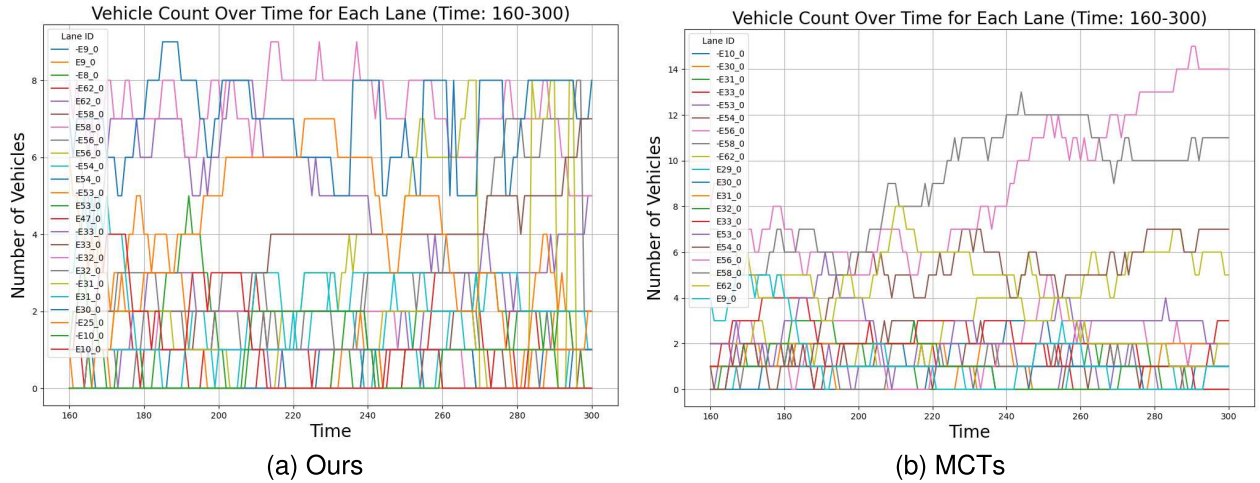


Fig. 11. Comparison of CAV counts on each lane over 140 steps using our method (a) and the MCTs algorithm (b).

TABLE III
PERFORMANCE COMPARISON BY VEHICLE QUANTITY

Vehicles Num	Computation Time (ms)		Objective Value	
	CPLEX	Hybrid Filtered Beam Search	CPLEX	Hybrid Filtered Beam Search
5	17.2	0.2	25.0	25.0
10	41.6	0.4	30.0	30.0
15	83.5	0.6	35.0	35.0
20	162.8	0.9	35.0	36.2

solution space. However, its computational cost grew exponentially with the increased number of vehicles. This complexity primarily stems from the quadratic growth of binary conflict

variables b_{ji} , which introduce $O(N^2)$ combinatorial constraints. For instance, at $N = 15$, the solver required 83.5 milliseconds, exceeding the practical time budget for millisecond-level decision-making.

In contrast, the Hybrid Filtered Beam Search method reduced the time complexity to a quasi-polynomial scale by pruning dominated states and pre-screening kinematic constraints. At $N = 20$, this approach maintained a solution time of 0.9 milliseconds with a bounded optimality gap.

H. Ablation Study

Table IV reports an ablation study. From the results, we found that if feasibility behavior correction is not introduced

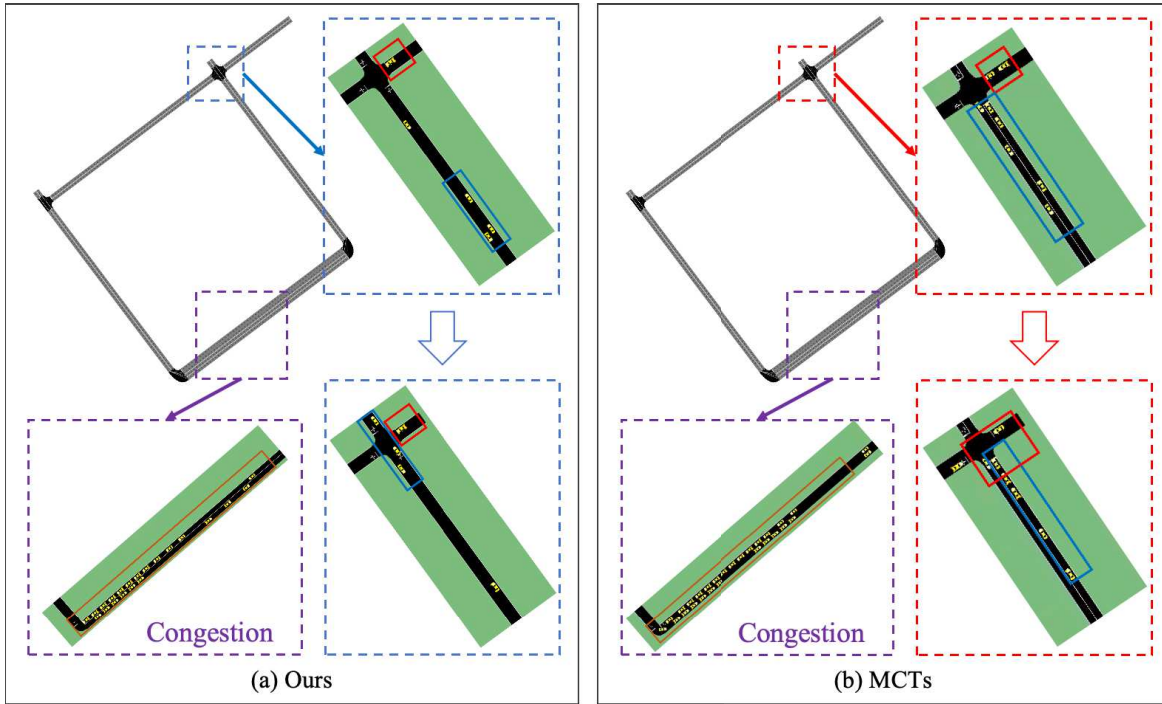


Fig. 12. A comparison of our method's decision outcomes with the optimal baseline MCTs in similar scenarios.

TABLE IV
ABLATION STUDY

Method	Total Delay Time	Total Waiting Time	Task Finished
Ours	182.987	215.650	613
w/o hard rule	269.833	289.883	551
w/o scheduling	206.805	235.266	595

during training, the overall algorithm cannot converge. Secondly, if only the results of planning are used to obtain lane-level priority scheduling results, performance will also be affected. This proves that if only global priorities are considered without local dynamic adjustment, it may lead to short-term congestion at some intersections not being promptly alleviated, thereby reducing the system's response efficiency.

I. Case Study

Additionally, we conducted a case study on scenarios with different vehicle quantity distributions. Fig. 12 illustrates two similar congestion situations and the different decision outcomes between our method and the single-intersection optimal passage method. From the results, it can be seen that our method takes into account the passage conditions of the next intersection, more reasonably distributes the priority of vehicles, and avoids exacerbating local congestion.

V. CONCLUSION AND FUTURE WORK

In this paper, we propose a novel two-stage decision-making framework for passage order, consisting of a planning stage and a scheduling stage. In the planning stage, we innovatively introduce a decision-making strategy that utilizes MAPPO to learn the priority relationships between vehicles.

In the scheduling stage, we use this priority relationship as a soft constraint to formulate a V2V interaction optimization problem and employ a hybrid filtered beam search algorithm to determine the final vehicle passage order. We conducted extensive experiments on datasets from various scenarios, and the results show that our method achieves state-of-the-art performance. This two-stage decision-making framework has already been deployed in an autonomous driving fleet at a port, where the planning stage currently employs an expert policy model for decision-making to bridge the sim-to-real gap, achieving satisfactory outcomes. We are presently collecting transport data from real-world scenarios and developing an offline reinforcement learning training method to create new strategies that will outperform expert-based approaches shortly.

REFERENCES

- [1] Q. Guo, L. Li, and X. J. Ban, "Urban traffic signal control with connected and automated vehicles: A survey," *Transp. Res. C, Emerg. Technol.*, vol. 101, pp. 313–334, Apr. 2019.
- [2] L. Li, D. Wen, and D. Yao, "A survey of traffic control with vehicular communications," *IEEE Trans. Intell. Transp. Syst.*, vol. 15, no. 1, pp. 425–432, Feb. 2014.
- [3] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artif. Intell.*, vol. 219, pp. 40–66, Feb. 2015.
- [4] E. Boyarsk et al., "ICBS: The improved conflict-based search algorithm for multi-agent pathfinding," in *Proc. Int. Symp. Combinat. Search*, 2015, vol. 6, no. 1, pp. 223–225.
- [5] G. Gange, D. Harabor, and P. J. Stuckey, "Lazy CBS: Implicit conflict-based search using lazy clause generation," in *Proc. Int. Conf. Automated Planning Scheduling*, vol. 29, 2019, pp. 155–162.
- [6] P. Li and X. Zhou, "Recasting and optimizing intersection automation as a connected-and-automated-vehicle (CAV) scheduling problem: A sequential branch-and-bound search approach in phase-time-traffic hypernetwork," *Transp. Res. B, Methodol.*, vol. 105, pp. 479–506, Nov. 2017.

- [7] D. Fan and P. Shi, "Improvement of Dijkstra's algorithm and its application in route planning," in *Proc. 7th Int. Conf. Fuzzy Syst. Knowl. Discovery*, vol. 4, Aug. 2010, pp. 1901–1904.
- [8] S. Erke, D. Bin, N. Yiming, Z. Qi, X. Liang, and Z. Dawei, "An improved A-star based path planning algorithm for autonomous land vehicles," *Int. J. Adv. Robotic Syst.*, vol. 17, no. 5, Sep. 2020, Art. no. 1729881420962263.
- [9] T. Sukuvaara and P. Nurmi, "Wireless traffic service platform for combined vehicle-to-vehicle and vehicle-to-infrastructure communications," *IEEE Wirel. Commun.*, vol. 16, no. 6, pp. 54–61, Dec. 2009.
- [10] J. Ding, L. Li, H. Peng, and Y. Zhang, "A rule-based cooperative merging strategy for connected and automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 8, pp. 3436–3446, Aug. 2020.
- [11] S. Ilgin Guler, M. Menendez, and L. Meier, "Using connected vehicle technology to improve the efficiency of intersections," *Transp. Res. C, Emerg. Technol.*, vol. 46, pp. 121–131, Sep. 2014.
- [12] L. Li and F.-Y. Wang, "Cooperative driving at blind crossings using intervehicle communication," *IEEE Trans. Veh. Technol.*, vol. 55, no. 6, pp. 1712–1724, Nov. 2006.
- [13] G. Liu, F. Dai, B. Huang, Z. Qiang, S. Wang, and L. Li, "A collaborative computation and dependency-aware task offloading method for vehicular edge computing: A reinforcement learning approach," *J. Cloud Comput.*, vol. 11, no. 1, p. 68, Oct. 2022.
- [14] S.-H. Hu, Q.-Y. Luo, G.-H. Li, W. Shi, and B.-L. Ye, "CA-DTS: A distributed and collaborative task scheduling algorithm for edge computing enabled intelligent road network," *J. Comput. Sci. Technol.*, vol. 38, no. 5, pp. 1113–1131, Sep. 2023.
- [15] Y. Zhan, S. Guo, P. Li, and J. Zhang, "A deep reinforcement learning based offloading game in edge computing," *IEEE Trans. Comput.*, vol. 69, no. 6, pp. 883–893, Jun. 2020.
- [16] X. Huang, S. Leng, S. Maharjan, and Y. Zhang, "Multi-agent deep reinforcement learning for computation offloading and interference coordination in small cell networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9282–9293, Sep. 2021.
- [17] C. Yu, A. Velu, E. Vinitzky, Y. Wang, A. M. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative, multi-agent games," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 24611–24624.
- [18] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. S. Zemel, "Neural relational inference for interacting systems," in *Proc. Int. Conf. Mach. Learn.*, vol. 80, 2018, pp. 2688–2697. [Online]. Available: <https://api.semanticscholar.org/CorpusID:215716912>
- [19] S. Hsaini, R. Ammour, L. Brenner, M. E. H. Charaf, and I. Demongodin, "A decentralized based approach using hybrid filtered beam search algorithm for monitoring patrols," in *Proc. 9th Int. Conf. Control, Decis. Inf. Technol. (CoDIT)*, Jul. 2023, pp. 1436–1441.
- [20] G. Mejía and K. Niño, "A new hybrid filtered beam search algorithm for deadlock-free scheduling of flexible manufacturing systems using Petri nets," *Comput. Ind. Eng.*, vol. 108, pp. 165–176, Jun. 2017.
- [21] S. Feng, Y. Zhang, S. E. Li, Z. Cao, H. Liu, and L. Li, "String stability for vehicular platoon control: Definitions and analysis methods," *Annu. Rev. Control*, vol. 47, pp. 81–97, Mar. 2019.
- [22] B. Xu, X. J. Ban, Y. Bian, J. Wang, and K. Li, "V2I based cooperation between traffic signal and approaching automated vehicles," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2017, pp. 1658–1664.
- [23] R. Tachet et al., "Revisiting street intersections using slot-based systems," *PLoS ONE*, vol. 11, no. 3, Mar. 2016, Art. no. e0149607.
- [24] Y. Meng, L. Li, F.-Y. Wang, K. Li, and Z. Li, "Analysis of cooperative driving strategies for nonsignalized intersections," *IEEE Trans. Veh. Technol.*, vol. 67, no. 4, pp. 2900–2911, Apr. 2018.
- [25] H. Xu, S. Feng, Y. Zhang, and L. Li, "A grouping-based cooperative driving strategy for CAVs merging problems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 6, pp. 6125–6136, Jun. 2019.
- [26] K. Dresner and P. Stone, "Multiagent traffic management: A reservation-based intersection control mechanism," in *Proc. 3rd Int. Joint Conf. Auto. Agents Multiagent Syst.*, vol. 3, 2004, pp. 530–537.
- [27] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *J. Artif. Intell. Res.*, vol. 31, pp. 591–656, Mar. 2008.
- [28] E. R. Müller, R. C. Carlson, and W. K. Junior, "Intersection control for automated vehicles with MILP," *IFAC-PapersOnLine*, vol. 49, no. 3, pp. 37–42, 2016.
- [29] X. Pan, B. Chen, L. Dai, S. Timotheou, and S. A. Evangelou, "A hierarchical robust control strategy for decentralized signal-free intersection management," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 5, pp. 2011–2026, Sep. 2023.
- [30] L. Chen and C. Englund, "Cooperative intersection management: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 2, pp. 570–586, Feb. 2016.
- [31] H. Xu, Y. Zhang, L. Li, and W. Li, "Cooperative driving at unsignalized intersections using tree search," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 11, pp. 4563–4571, Nov. 2020.
- [32] C. Chen et al., "A graph-based conflict-free cooperation method for intelligent electric vehicles at unsignalized intersections," in *Proc. IEEE Int. Intell. Transp. Syst. Conf. (ITSC)*, Sep. 2021, pp. 52–57.
- [33] J. Luo, T. Zhang, and Q. Zhang, "A computationally efficient bi-level coordination framework for CAVs at unsignalized intersections," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 1868–1878, Feb. 2024.
- [34] M. Enzenberger, M. Müller, B. Arneson, and R. Segal, "Fuego—An open-source framework for board games and go engine based on Monte Carlo tree search," *IEEE Trans. Comput. Intell. AI Games*, vol. 2, no. 4, pp. 259–270, Dec. 2010.
- [35] D. Silver et al., "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, Oct. 2017.
- [36] Y. Shi, W. Wang, X. Tao, I. Dusparic, and V. Cahill, "Applying neural Monte Carlo tree search to unsignalized multi-intersection scheduling for autonomous vehicles," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 935–942.
- [37] A. Uno, T. Sakaguchi, and S. Tsugawa, "A merging control algorithm based on inter-vehicle communication," in *Proc. IEEE/IEEEJ/ISAI Int. Conf. Intell. Transp. Syst.*, Oct. 1999, pp. 783–787.
- [38] R. Zhao, Y. Li, K. Wang, Y. Fan, F. Gao, and Z. Gao, "Centralized cooperation for connected autonomous vehicles at intersections by safe deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 12830–12847, Dec. 2024.
- [39] Y. Guan, Y. Ren, S. E. Li, Q. Sun, L. Luo, and K. Li, "Centralized cooperation for connected and automated vehicles at intersections by proximal policy optimization," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 12597–12608, Nov. 2020.
- [40] J. Li, P. Surynek, A. Felner, H. Ma, T. S. Kumar, and S. Koenig, "Multi-agent path finding for large agents," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 1, pp. 7627–7634.
- [41] G. Sharon, R. Stern, M. Goldenberg, and A. Felner, "The increasing cost tree search for optimal multi-agent pathfinding," *Artif. Intell.*, vol. 195, pp. 470–495, Feb. 2013.
- [42] J. Li, Z. Chen, D. Harabor, P. J. Stuckey, and S. Koenig, "MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 9, pp. 10256–10265.
- [43] G. Sartoretti et al., "PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2378–2385, Jul. 2019.
- [44] Q. Li, W. Lin, Z. Liu, and A. Prorok, "Message-aware graph attention networks for large-scale multi-robot path planning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5533–5540, Jul. 2021.
- [45] B. Wang, Z. Liu, Q. Li, and A. Prorok, "Mobile robot path planning in dynamic environments through globally guided reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6932–6939, Oct. 2020.
- [46] W. Othman and N. Shilov, "Deep reinforcement learning for path planning by cooperative robots: Existing approaches and challenges," in *Proc. 28th Conf. Open Innov. Assoc. (FRUCT)*, Jan. 2021, pp. 349–357.
- [47] J. Ren and D. Xia, *Autonomous Driving Algorithms and Its IC Design*. Springer, 2023.
- [48] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
- [49] M. Hessel, H. Soyer, L. Espeholt, W. Czarnecki, S. Schmitt, and H. Van Hasselt, "Multi-task deep reinforcement learning with popart," in *Proc. AAAI Conf. Artif. Intell.*, May 2019, vol. 33, no. 1, pp. 3796–3803.
- [50] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Nov. 2018, pp. 2575–2582.
- [51] H. Xu, C. G. Cassandras, L. Li, and Y. Zhang, "Comparison of cooperative driving strategies for CAVs at signal-free intersections," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 7614–7627, Jul. 2022.
- [52] Y. Zhang and C. G. Cassandras, "A decentralized optimal control framework for connected automated vehicles at urban intersections with dynamic resequencing," in *Proc. IEEE Conf. Decis. Control (CDC)*, Dec. 2018, pp. 217–222.



Weihang Pan is currently pursuing the Ph.D. degree with the School of Software, Zhejiang University. His research interests include reinforcement learning, decision making, and intelligent transportation systems.

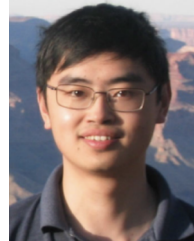


distributed systems.

Xinkui Zhao is a ZJU 100-Young Professor with the School of Software Technology, Zhejiang University, China. He has led the development of multiple large-scale, enterprise-level cloud-native platforms and has authored more than 30 academic papers published in premier journals and conferences such as ASPLOS, DAC, WWW, NeurIPS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, and IEEE TRANSACTIONS ON SERVICES COMPUTING. His research interests primarily focus on intelligent agents and intelligent



Binbin Lin received the Ph.D. degree in computer science from Zhejiang University, China, in 2012. He is an Assistant Professor with the School of Software Technology, Zhejiang University. His research interests include machine learning and decision making.



Xiaofei He (Senior Member, IEEE) received the B.S. degree in computer science from Zhejiang University, China, in 2000, and the Ph.D. degree in computer science from The University of Chicago in 2005. He was a Research Scientist with Yahoo! Research Labs, Burbank, CA, USA. He is currently a Professor with the State Key Laboratory of CAD&CG, Zhejiang University. His research interests include machine learning, information retrieval, and computer vision.



Yafei Wang is currently pursuing the master's degree with the Polytechnic Institute, Zhejiang University. Her research interests include reinforcement learning, data-driven intelligent decision-making, and operations optimization.



Jieping Ye is currently the Vice President of Alibaba Group, Hangzhou, China, where he is the Head of the CityBrain Lab, DAMO Academy. His research interests include big data, machine learning, artificial intelligence with applications in transportation, smart cities, and biomedicine. He was elevated to an IEEE Fellow in 2019 and named an ACM Distinguished Scientist in 2020 for his contributions to the methodology and application of machine learning and data mining. He received the NSF CAREER Award in 2010. His papers have been selected for the Outstanding Student Paper at the International Conference on Machine Learning (ICML) in 2004, the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD) Best Research Paper Runner Up in 2013, and the KDD Best Student Paper Award in 2014. He received First Place in the 2019 INFORMS Daniel H. Wagner Prize, one of the top awards in operations research practice. He has served as a Senior Program Committee/Area Chair/Program Committee Vice Chair of many conferences, including the Conference and Workshop on Neural Information Processing Systems (NeurIPS), ICML, KDD, International Joint Conference on Artificial Intelligence (IJCAI), IEEE International Conference on Data Mining (ICDM), and the SIAM International Conference on Data Mining (SDM). He has served as an Associate Editor for *Data Mining and Knowledge Discovery*, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, and IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE.



Zhengxu Yu received the Ph.D. degree from Zhejiang University in March 2021. He is currently an Algorithm Engineer with Alibaba Cloud. His research interests include large-scale machine learning and decision making.